

Expression Bodied Members in C# 7 with Examples

In this article, I am going to discuss the **Expression Bodied Members in C# 7** with Examples. Please read our previous article where we discussed the [Generalized Async Return Types in C#](#) with Examples. At the end of this article, you will understand what exactly Expression Bodied Members in C# are and when and how to use this in C# with examples.

What are Expression Bodied Members in C#?

The Expression Bodied Members in C# 7, allows us to provide the implementation of a member in a better readable format. You can use Expression Bodied Members in C# whenever the logic for any supported members such as a method or property consists of a single expression. An Expression Bodied Member definition has the following general syntax:

member => expression; Where expression is a valid C# expression.

The Expression Bodied Members was first introduced in C# 6 with only Methods and Properties. But with C# 7, several new members have been included in the list. The complete list of Expression Bodied Members is as follows.

1. Methods
2. Properties
3. Constructor
4. Destructor
5. Getters
6. Setters
7. Indexers

Let's discuss each of these members one by one with examples.

Method Expression Bodied Members in C#:

An Expression Bodied Method consists of a single expression that returns a value whose type matches the method's return type, or, for methods that return void, that performs some operation. For example, types that override the ToString method typically include a single expression that returns the string representation of the current object.

For a better understanding, please have a look at the following example. In the below example, we define the Employee class that overrides the ToString method with an Expression Body Definition. It also defines the GetFullName method that returns the full name of the employee and again it also defines a DisplayName method that displays the full name on the console. Note that the return keyword is not used in the ToString as well as with the GetFullName Expression Bodied Definitions. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
    {
```

```

static void Main()
{
    Employee employee = new Employee("Pranaya", "Rout");
    employee.DisplayName();
    Console.WriteLine(employee); //It will Internally call the ToString()
    Console.WriteLine("Press any key to exists");
    Console.ReadKey();
}
}
public class Employee
{
    private string FirstName;
    private string LastName;
    public Employee(string firstName, string lastName)
    {
        FirstName = firstName;
        LastName = lastName;
    }
    //The following GetFullName() Method contains a single statement
    //public string GetFullName()
    //{
    //    return $"{FirstName} {LastName}";
    //}
    //The above GetFullName() Method can be rewritten using Expression Bodier
as follows
    public string GetFullName() => $"{FirstName} {LastName}";
    //The following Override ToString() Method contains a single statement
    //public override string ToString()
    //{
    //    return $"{FirstName} {LastName}";
    //}
    //The above ToString() Method can be rewritten using Expression Bodier
follows
    public override string ToString() => $"{FirstName} {LastName}";
    //The following DisplayName Method contains a single statement
    //public void DisplayName()
    //{
    //    Console.WriteLine(GetFullName());
    //}
    //The above DisplayName() Method can be rewritten using Expression Bodier
as follows
    public void DisplayName() => Console.WriteLine(GetFullName());
}
}

```

Output:

```

Pranaya Rout
Pranaya Rout
Press any key to exists

```

Note: This removes not only the curly brackets but also the return keyword. Return is implicit with a lambda expression.

Generally, Expression Bodied Methods are more used than other members. They have the following characteristics.

1. Expression Bodied Methods can specify all the accessibility operators i.e. public, protected, internal, private, and protected internal.
2. These can be declared virtual or abstract or can even override a base class method.
3. Such methods can be static.
4. Methods can even support asynchronous behavior if they return void, Task, or Task<T>.

Constructors Expression Bodied Members in C#:

An expression body definition for a constructor typically consists of a single assignment expression or a method call that handles the constructor's arguments or initializes the instance members of the class. For a better understanding of the Constructors Expression Bodied Members in C#, please have a look at the following example. The following example defines a Location class whose constructor has a single string parameter named Name. The expression body definition assigns the argument to the Name property. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
    {
        static void Main()
        {
            Location location = new Location("Mumbai");
            Console.WriteLine(location.LocationName);
            Console.WriteLine("Press any key to exists");
            Console.ReadKey();
        }
    }
    public class Location
    {
        //Public property to hold the Location Name
        public string LocationName { get; set; }
        //The following Constructor Initialzie the LocationName property
        //public Location(string name)
        //{
        //    LocationName = name;
        //}
        //As the above Constructor contains a single statement, so we can rew
above
        //constructor using the Expression Bodied as follows
        public Location(string name) => LocationName = name;
    }
}
```

Output:

```
Mumbai
Press any key to exists
```

Destructors Expression Bodied Member in C#:

In C#, an expression body definition for a destructor typically contains cleanup statements, such as statements that release unmanaged resources. For a better understanding of the Destructor Expression Body Member in C#, please have a look at the following example. In the following example, we define a destructor that uses an expression body definition to indicate that the destructor has been called. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Main Method Exceution Started...");
            //Creating an Instance of Destructor Class
            Destructor destructor = new Destructor();
            //Making the destructor instance as Null so that it can be Garbage
            destructor = null;
            //Calling the GC.Collect() method to request garbage collector to
            unused objects
            GC.Collect();
            Console.WriteLine("Main Method Exceution Completed...");
            Console.ReadKey();
        }
    }
    public class Destructor
    {
        //The following Destructor Contains a Single Statement
        //~Destructor()
        //{
        //    Console.WriteLine($"The Destructor is Executing.");
        //}
        //We can write the above Destructor using Expression Bodied as follows
        //~Destructor() => Console.WriteLine($"The Destructor is Executing.");
    }
}
```

Output:

```
Main Method Execution Started...
Main Method Execution Completed...
The Destructor is Executing.
```

Property Expression Bodied Member in C#

If you choose to implement a property get or set accessor yourself, you can use an expression body definition for single expressions that simply return the property value or set the property value. Note that the return statement isn't used. The following example defines the LocationName property whose property Get accessor returns the value of the private `_locationName` field and the Set accessor of the LocationName property set the private `_locationName` field value using Expression Bodied Member. The following example code is self-explained, so please go through the comment line for a better understanding.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
    {
        static void Main()
        {
            //Setting _locationName using Constructor
            Location location = new Location("Mumbai");
            //Accessing the Location Name using LocationName Property
            Console.WriteLine(location.LocationName);
            //Setting the Location Name using LocationName Property
            location.LocationName = "Delhi";
            //Accessing the Location Name using LocationName Property
            Console.WriteLine(location.LocationName);
            Console.ReadKey();
        }
    }
    public class Location
    {
        //Private Variable to store the Location Name
        private string _locationName;
        //Constructor Expression Bodied Member to set the private variable _locationName
        public Location(string name) => _locationName = name;
        //You can also use the following to set the public property LocationName
        //public Location(string name) => LocationName = name;
        //Property Expression Bodied Member
        public string LocationName
        {
            get => _locationName;
            set => _locationName = value;
        }
    }
}
```

Read-Only properties that use an expression body definition can be implemented without an explicit set statement. The following example defines a Location class whose read-only LocationName property is implemented as an expression body definition that returns the value of the private `_locationName` field.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
    {
        static void Main()
        {
            //Setting _locationName using Constructor
            Location location = new Location("Mumbai");
            //Accessing the Location Name using LocationName Property
            Console.WriteLine(location.LocationName);
            //As the Property is Read-Only, you cannot set the Location Name u
            LocationName Property
            //location.LocationName = "Delhi";

            Console.ReadKey();
        }
    }
    public class Location
    {
        //Private Variable to store the Location Name
        private string _locationName;
        //Constructor Expression Bodied Member to set the private variable _lc
        public Location(string name) => _locationName = name;
        //You can also use the following to set the public property LocationNa
        //public Location(string name) => LocationName = name;
        //Expression Bodied Member for Read-Only Property
        public string LocationName => _locationName;
    }
}
```

The above code not only reduces the curly bracket but also writes the get accessor is not necessary.

Indexers Expression Bodied Member in C#

Like properties, an indexer's Get and Set accessors consist of expression body definitions if the get accessor consists of a single statement that returns a value or the set accessor performs a simple assignment. The following example defines a class named Sports that includes an internal String array that contains the names of a number of sports. Both the indexer's get and set accessors are implemented as expression body definitions. The following example code is self-explained, so please go through the comment line for a better understanding.

```
using System;
namespace ExpressionBodiedMembersDemo
{
    class Program
```

```

{
    static void Main()
    {
        //Create an Instance of the Sports Class
        Sports sports = new Sports();
        //Accessing the Sports Elements using Integer Indexers like an Array
        Console.WriteLine("Accessing Sports Elements sports[0] & sports[2]");
        Console.WriteLine($"sports[0] : {sports[0]}");
        Console.WriteLine($"sports[2] : {sports[2]}");
        //Updating Sports Elements using Indexer
        Console.WriteLine("Updating Sports Elements sports[0] & sports[2]");
        sports[0] = "Cricket Updated";
        sports[2] = "Basketball Updated";
        //Accessing the Sports Elements using Integer Indexers like an Array
        Console.WriteLine("Accessing Sports Elements sports[0] & sports[2]");
        Console.WriteLine($"sports[0] : {sports[0]}");
        Console.WriteLine($"sports[2] : {sports[2]}");
        Console.ReadKey();
    }
}

public class Sports
{
    //String Array to Store list of Sports
    private readonly string[] SportTypes = { "Cricket", "Baseball", "Basketball", "Football", "Hockey", "Soccer", "Tennis", "Volleyball" };
    //Creating an Indexer which will return the string data based on the Index Position
    public string this[int i]
    {
        //Get Accessor Implemented as Expression Bodied Member
        get
        {
            return SportTypes[i];
        }
        //Set Accessor Implemented as Expression Bodied Member
        set
        {
            SportTypes[i] = value;
        }
    }
    //You can rewrite the above Indexer using Indexer Expression Bodied Member
    public string this[int i]
    {
        //Get Accessor Implemented as Expression Bodied Member
        get => SportTypes[i];
        //Set Accessor Implemented as Expression Bodied Member
        set => SportTypes[i] = value;
    }
}
}

```

Output:

```
Accessing Sprots Elements sports[0] & sports[2]
sports[0] : Cricket
sports[2] : Basketball
Updating Sprots Elements sports[0] & sports[2]
Accessing Sprots Elements sports[0] & sports[2]
sports[0] : Cricket Updated
sports[2] : Basketball Updated
```

Getter and Setter Expression Bodied Members in C#

Expression body getters and setters are also introduced in C# 7.0. They allow an expression to be used in the body of getters or setters. The example given below illustrates the same.

```
class Program
{
    static void Main(string[] args)
    {
        var obj = new ExprBodiedGettersnSetters();
        obj.EmpBasicSalaryList.Add(101, 1000);
        obj.EmpBasicSalaryList.Add(102, 1200);
        obj.EmpId = 101;
        Console.WriteLine($"The basic salary of EmpId {obj.EmpId} is:
{obj.EmpBasicSalary}");
        obj.EmpBasicSalary = 1500;
        Console.WriteLine($"The updated basic salary of EmpId {obj.EmpId} is:
{obj.EmpBasicSalary}");
        Console.WriteLine("Press any key to exist.");
        Console.ReadKey();
    }
}

class ExprBodiedGettersnSetters
{
    public Dictionary<int, double> EmpBasicSalaryList = new Dictionary<int, double>();
    public int EmpId { get; set; }
    public double EmpBasicSalary
    {
        ///Expression Bodied Getter
        get => EmpBasicSalaryList[EmpId];
        ///Expression Bodied Setter
        set => EmpBasicSalaryList[EmpId] = value;
    }
}
```

Output:

```
The basic salary of EmpId 101 is: 1000
The updated basic salary of EmpId 101 is: 1500
Press any key to exist.
```

Limitations of Expression Bodied Members in C#

Although Expression Bodied Members in C# 7 provide very clean syntax, they have also some limitations. Let's go through some of them and see how those can be addressed.

The Expression Bodied Members in C# 7 don't support a block of code. It supports only one statement with an expression. If you need to use more than one statement then you may use the regular methods or regular properties but not the Expression Bodied Members.

Branching Statements such as (if..else, switch case) are not allowed with Expression Bodied Members. However if..else behavior can be addressed by using the ternary operator. For example, the statement given below can work.

```
public string FullName() => (middleName != null) ? firstName + " " + middleName + " " + lastName :
firstName + " " + lastName;
```

Looping statements such as (for, for each, while, and do..while are not allowed) with Expression Bodied Members in C# 7. However, in some cases, it can be addressed with LINQ queries. For example, both of the following methods (HundredNumbersList and HundredNumbersListWithExprBody) return the same result.

Using Regular Method:

```
public IEnumerable<int> HundredNumbersList()
{
    for (int i = 0; i < 100; i++)
        yield return i;
}
```

Using Expression Bodied Members with LINQ:

```
public IEnumerable<int> HundredNumbersListWithExprBody() => from n in Enumerable.Range(0,
100)
select n;
```

Note:

Lambda Expressions can be written in short form without curly brackets when the statement consists of a single line. Lambda expressions can also be written in the long form where curly brackets and the return statement are needed. This longer syntax is not possible with Expression Bodied Members in C#. If one code line is insufficient, you can use the normal syntax with curly brackets, as has been available since C# 1.0.

In the next article, I am going to discuss [Throw Expressions in C#](#) with Examples. Here, In this article, I try to explain **Expression Bodied Members in C#** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.